Department of Computer Science and Engineering

Institute of Science and Technology, National University

Submitted To	
Teacher Name	**Tasmi Sultana**
Designation	**Lecturer**
Course Title	**Computer Graphics Lab**
Course Code	**540206**

Submitted By	
Name of Examinee	**Md. Maruf Ahmed**
Registration No.	**2050200461**
Class Roll No.	**21056**
Date of Submission	**14/06/2026**
Session	**2020-21**

I hereby declare that the work represented here is not copied from anywhere else and totally accomplished by myself. If proven otherwise, I will accept the consequences.

---------- ----------

Signature of Student Signature of Course Teacher

Index of Experiments

Experiment No. 1

Name of the Problem: DDA (Digital Differential Analyzer) Line Drawing Algorithm

Source Code:

```
import matplotlib.pyplot as plt

x1, y1 = 2, 3
x2, y2 = 18, 13

dx = x2 - x1
dy = y2 - y1
steps = max(abs(dx), abs(dy))

x_inc = dx / steps
y_inc = dy / steps

x = x1
y = y1
points = []

for i in range(steps + 1):
   points.append((round(x), round(y)))
   x += x_inc
   y += y_inc

xs, ys = zip(*points)

plt.figure(figsize=(16, 9))
plt.scatter(xs, ys, s=90)
plt.plot(xs, ys)
plt.title("DDA Line Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.savefig("01_dda_line_output.png", dpi=120)
plt.show()
```

Output Screenshot:

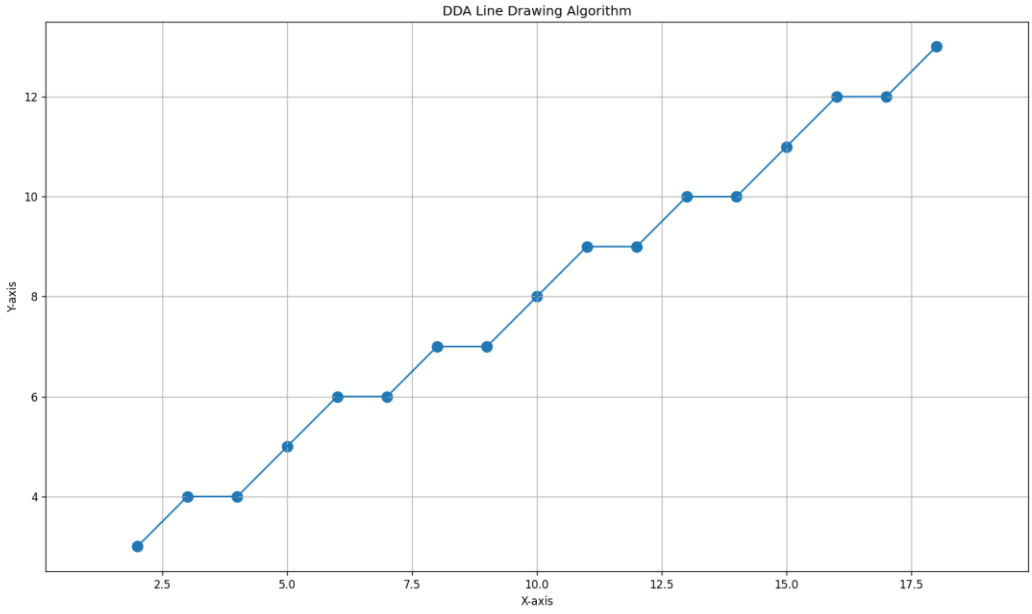

Figure 1: Output of DDA (Digital Differential Analyzer) Line Drawing Algorithm

Experiment No. 2

Name of the Problem: Bresenham's Line Drawing Algorithm

Source Code:

```
import matplotlib.pyplot as plt

x1, y1 = 2, 2
x2, y2 = 17, 10

points = []
dx = abs(x2 - x1)
dy = abs(y2 - y1)
sx = 1 if x1 < x2 else -1
sy = 1 if y1 < y2 else -1
err = dx - dy
x, y = x1, y1

while True:
    points.append((x, y))
    if x == x2 and y == y2:
        break
    e2 = 2 * err
    if e2 > -dy:
        err -= dy
        x += sx
    if e2 < dx:
        err += dx
        y += sy

xs, ys = zip(*points)

plt.figure(figsize=(16, 9))
plt.scatter(xs, ys, s=90)
plt.plot(xs, ys)
plt.title("Bresenham Line Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.savefig("02_bresenham_line_output.png", dpi=120)
plt.show()
```

Output Screenshot:

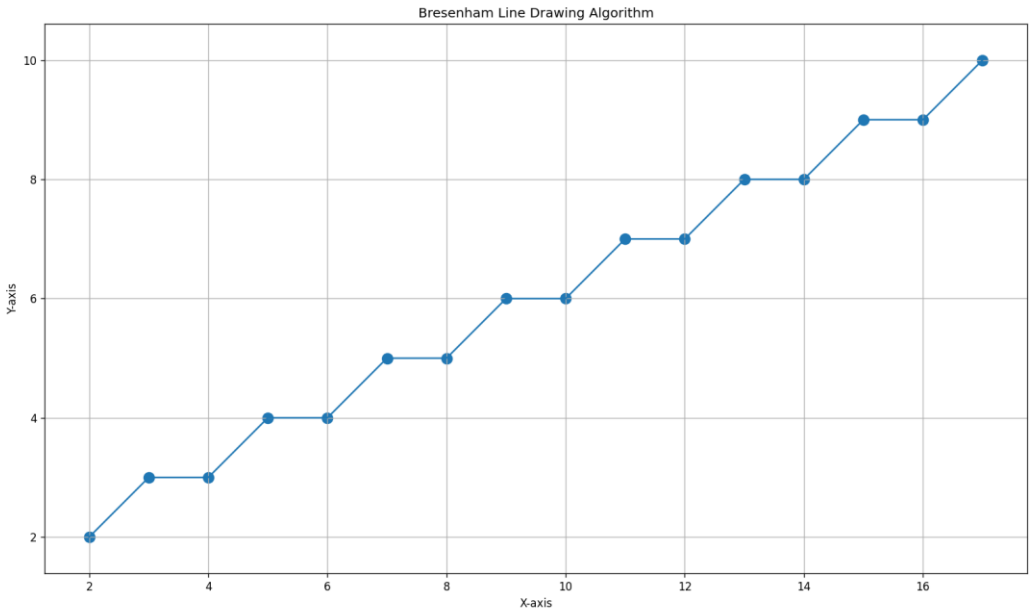

Figure 2: Output of Bresenham's Line Drawing Algorithm

Experiment No. 3

Name of the Problem: Bresenham's Circle Drawing Algorithm

Source Code:

```
import matplotlib.pyplot as plt

xc, yc = 0, 0
r = 8
points = []

def add_points(x, y):
    points.extend([
        (xc + x, yc + y), (xc - x, yc + y),
        (xc + x, yc - y), (xc - x, yc - y),
        (xc + y, yc + x), (xc - y, yc + x),
        (xc + y, yc - x), (xc - y, yc - x)
    ])

x = 0
y = r
d = 3 - 2 * r

while x <= y:
    add_points(x, y)
    if d < 0:
        d = d + 4 * x + 6
    else:
        d = d + 4 * (x - y) + 10
        y -= 1
    x += 1

xs, ys = zip(*points)

plt.figure(figsize=(16, 9))
plt.scatter(xs, ys, s=90)
plt.title("Bresenham Circle Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.savefig("03_bresenham_circle_output.png", dpi=120)
plt.show()
```

Output Screenshot:

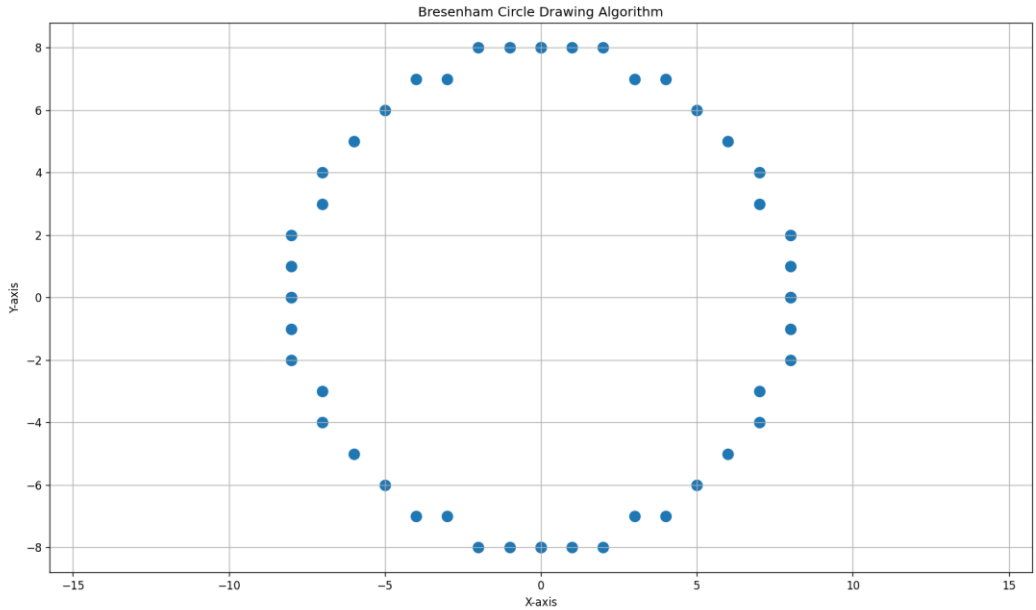

Figure 3: Output of Bresenham's Circle Drawing Algorithm

Experiment No. 4

Name of the Problem: Midpoint Circle Drawing Algorithm

Source Code:

```
import matplotlib.pyplot as plt

xc, yc = 0, 0
r = 9
points = []

def add_points(x, y):
    points.extend([
        (xc + x, yc + y), (xc - x, yc + y),
        (xc + x, yc - y), (xc - x, yc - y),
        (xc + y, yc + x), (xc - y, yc + x),
        (xc + y, yc - x), (xc - y, yc - x)
    ])

x = 0
y = r
p = 1 - r

while x <= y:
    add_points(x, y)
    if p < 0:
        p = p + 2 * x + 3
    else:
        p = p + 2 * (x - y) + 5
        y -= 1
    x += 1

xs, ys = zip(*points)

plt.figure(figsize=(16, 9))
plt.scatter(xs, ys, s=90)
plt.title("Midpoint Circle Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.savefig("04_midpoint_circle_output.png", dpi=120)
plt.show()
```

Output Screenshot:

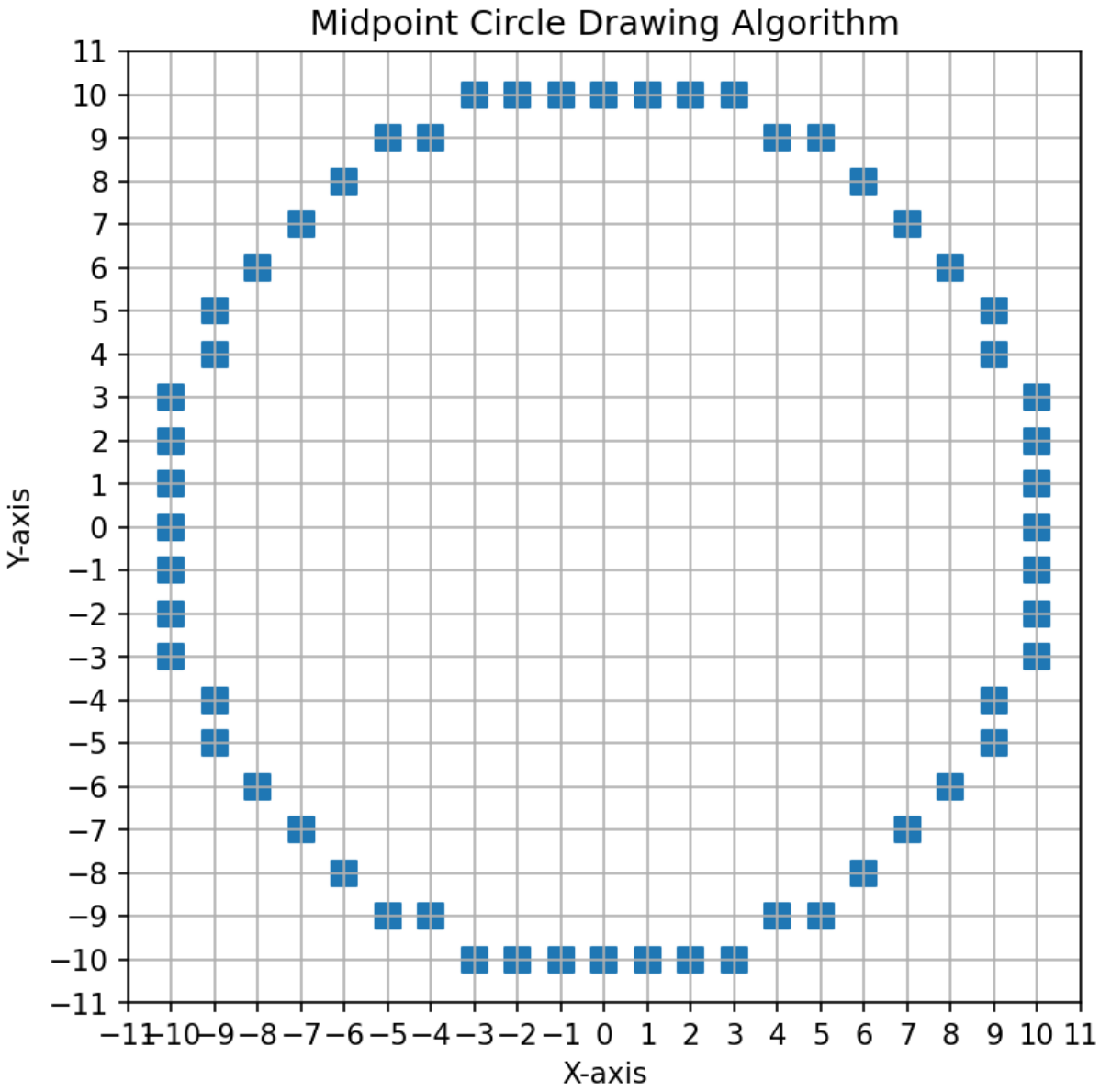

Figure 4: Output of Midpoint Circle Drawing Algorithm

Experiment No. 5

Name of the Problem: Midpoint Ellipse Drawing Algorithm

Source Code:

```
import matplotlib.pyplot as plt

xc, yc = 0, 0
rx, ry = 10, 6
points = []

def add_points(x, y):
    points.extend([
        (xc + x, yc + y), (xc - x, yc + y),
        (xc + x, yc - y), (xc - x, yc - y)
    ])

x = 0
y = ry
rx2 = rx * rx
ry2 = ry * ry
dx = 2 * ry2 * x
dy = 2 * rx2 * y
p1 = ry2 - (rx2 * ry) + (0.25 * rx2)

while dx < dy:
    add_points(x, y)
    if p1 < 0:
        x += 1
        dx = dx + 2 * ry2
        p1 = p1 + dx + ry2
    else:
        x += 1
        y -= 1
        dx = dx + 2 * ry2
        dy = dy - 2 * rx2
        p1 = p1 + dx - dy + ry2

p2 = (ry2 * (x + 0.5) * (x + 0.5)) + (rx2 * (y - 1) * (y - 1)) - (rx2 * ry2)

while y >= 0:
    add_points(x, y)
    if p2 > 0:
        y -= 1
        dy = dy - 2 * rx2
        p2 = p2 + rx2 - dy
    else:
        y -= 1
        x += 1
        dx = dx + 2 * ry2
        dy = dy - 2 * rx2
        p2 = p2 + dx - dy + rx2

xs, ys = zip(*points)

plt.figure(figsize=(16, 9))
plt.scatter(xs, ys, s=90)
plt.title("Midpoint Ellipse Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.savefig("05_midpoint_ellipse_output.png", dpi=120)
plt.show()
```

Output Screenshot:

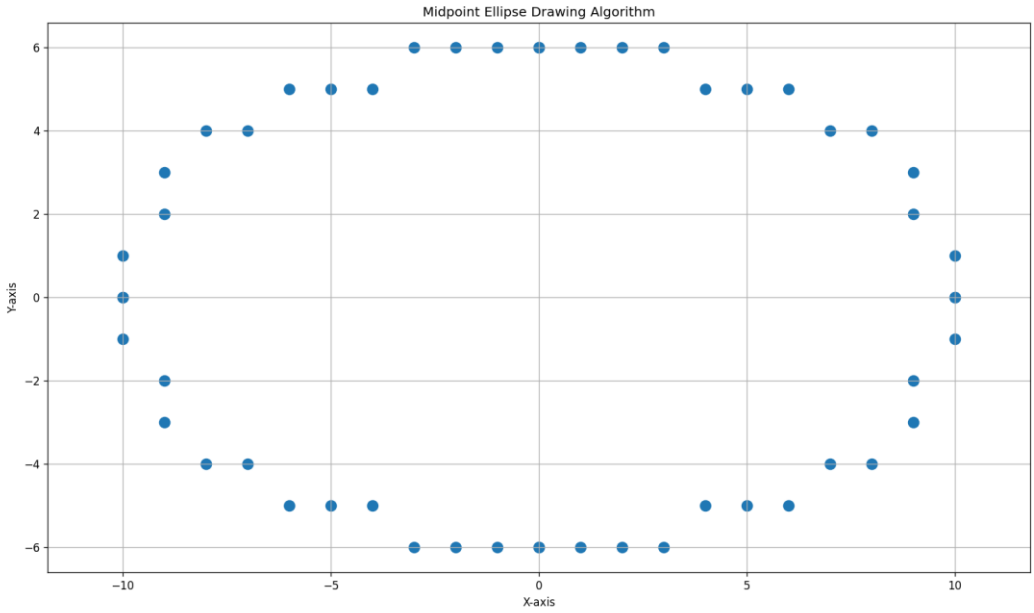

Figure 5: Output of Midpoint Ellipse Drawing Algorithm

Experiment No. 6

Name of the Problem: Polygon (Rectangle) Filling Algorithm

Source Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

x_min, y_min = 3, 3
x_max, y_max = 14, 9
filled = []

for y in range(y_min, y_max + 1):
    for x in range(x_min, x_max + 1):
        filled.append((x, y))

xs, ys = zip(*filled)

plt.figure(figsize=(16, 9))
plt.scatter(xs, ys, s=120)
plt.gca().add_patch(Rectangle((x_min, y_min), x_max - x_min, y_max - y_min, fill=False, linewidth=3))
plt.title("Polygon Rectangle Filling Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.savefig("06_rectangle_filling_output.png", dpi=120)
plt.show()
```

Output Screenshot:

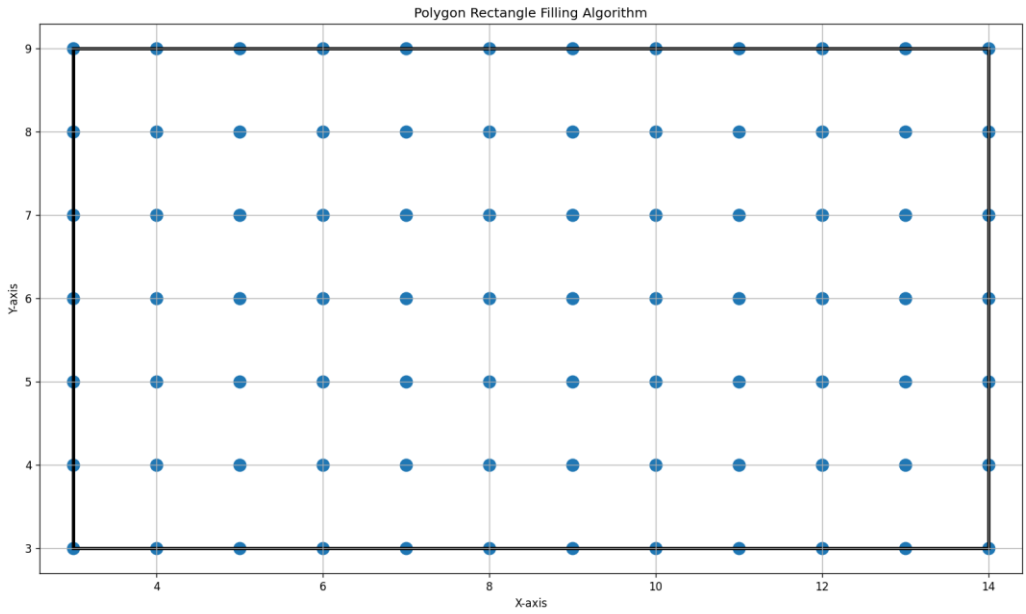

Figure 6: Output of Polygon (Rectangle) Filling Algorithm

Experiment No. 7

Name of the Problem: 2D Transformation (Translation, Rotation)

Source Code:

```
import math
import matplotlib.pyplot as plt

triangle = [(1, 1), (5, 1), (3, 4), (1, 1)]
tx, ty = 4, 2
angle = 45
rad = math.radians(angle)

translated = []
for x, y in triangle:
    translated.append((x + tx, y + ty))

rotated = []
for x, y in translated:
    xr = x * math.cos(rad) - y * math.sin(rad)
    yr = x * math.sin(rad) + y * math.cos(rad)
    rotated.append((xr, yr))

def draw(poly, label):
    xs = [p[0] for p in poly]
    ys = [p[1] for p in poly]
    plt.plot(xs, ys, marker="o", label=label)

plt.figure(figsize=(16, 9))
draw(triangle, "Original")
draw(translated, "After Translation")
draw(rotated, "After Translation and Rotation")
plt.title("2D Transformation: Translation and Rotation")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.savefig("07_translation_rotation_output.png", dpi=120)
plt.show()
```

Output Screenshot:

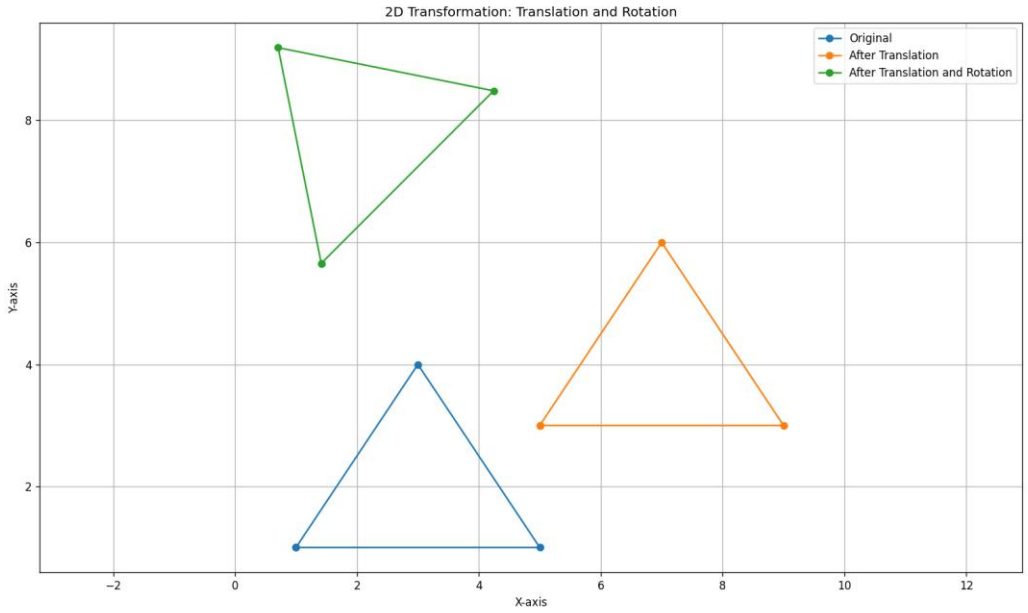

Figure 7: Output of 2D Transformation (Translation, Rotation)

Experiment No. 8

Name of the Problem: 2D Transformation (Scaling, Translation)

Source Code:

```
import matplotlib.pyplot as plt

rectangle = [(1, 1), (5, 1), (5, 3), (1, 3), (1, 1)]
sx, sy = 1.8, 1.5
tx, ty = 4, 2

scaled = []
for x, y in rectangle:
    scaled.append((x * sx, y * sy))

translated = []
for x, y in scaled:
    translated.append((x + tx, y + ty))

def draw(poly, label):
    xs = [p[0] for p in poly]
    ys = [p[1] for p in poly]
    plt.plot(xs, ys, marker="o", label=label)

plt.figure(figsize=(16, 9))
draw(rectangle, "Original")
draw(scaled, "After Scaling")
draw(translated, "After Scaling and Translation")
plt.title("2D Transformation: Scaling and Translation")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.savefig("08_scaling_translation_output.png", dpi=120)
plt.show()
```

Output Screenshot:

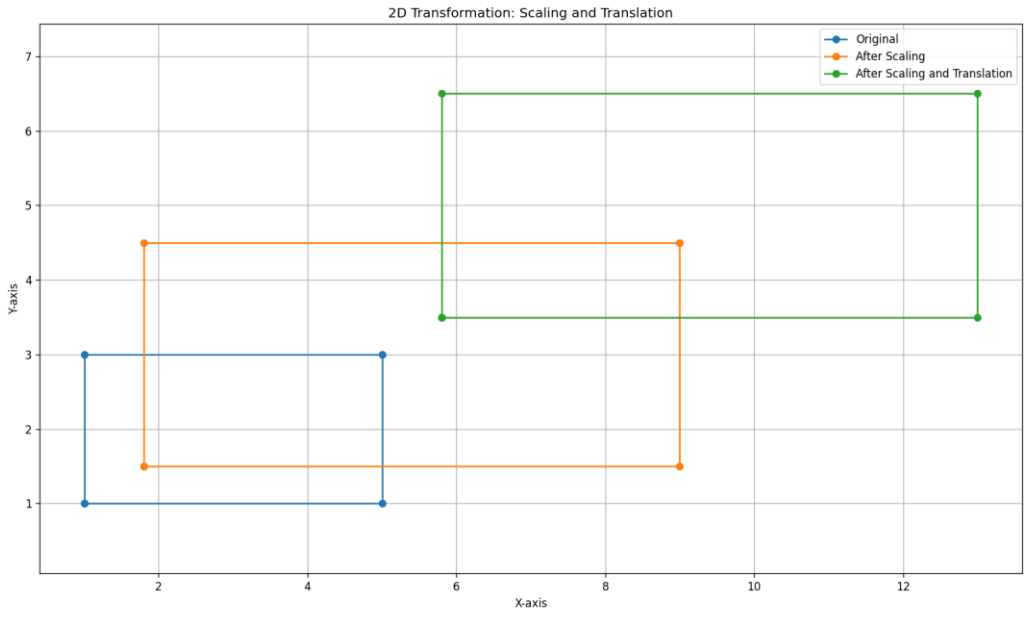

Figure 8: Output of 2D Transformation (Scaling, Translation)

Experiment No. 9

Name of the Problem: Composite (Translation, Rotation) Transformation

Source Code:

```
import math
import matplotlib.pyplot as plt

triangle = [(1, 1), (5, 1), (3, 4), (1, 1)]
tx, ty = 4, 2
angle = 60
rad = math.radians(angle)

T = [[1, 0, tx], [0, 1, ty], [0, 0, 1]]
R = [[math.cos(rad), -math.sin(rad), 0], [math.sin(rad), math.cos(rad), 0], [0, 0, 1]]

def multiply(A, B):
    result = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
    for i in range(3):
        for j in range(3):
            for k in range(3):
                result[i][j] += A[i][k] * B[k][j]
    return result

M = multiply(R, T)

composite = []
for x, y in triangle:
    nx = M[0][0] * x + M[0][1] * y + M[0][2]
    ny = M[1][0] * x + M[1][1] * y + M[1][2]
    composite.append((nx, ny))

def draw(poly, label):
    xs = [p[0] for p in poly]
    ys = [p[1] for p in poly]
    plt.plot(xs, ys, marker="o", label=label)

plt.figure(figsize=(16, 9))
draw(triangle, "Original")
draw(composite, "Composite Result")
plt.title("Composite Transformation: Translation and Rotation")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.savefig("09_composite_translation_rotation_output.png", dpi=120)
plt.show()
```

Output Screenshot:

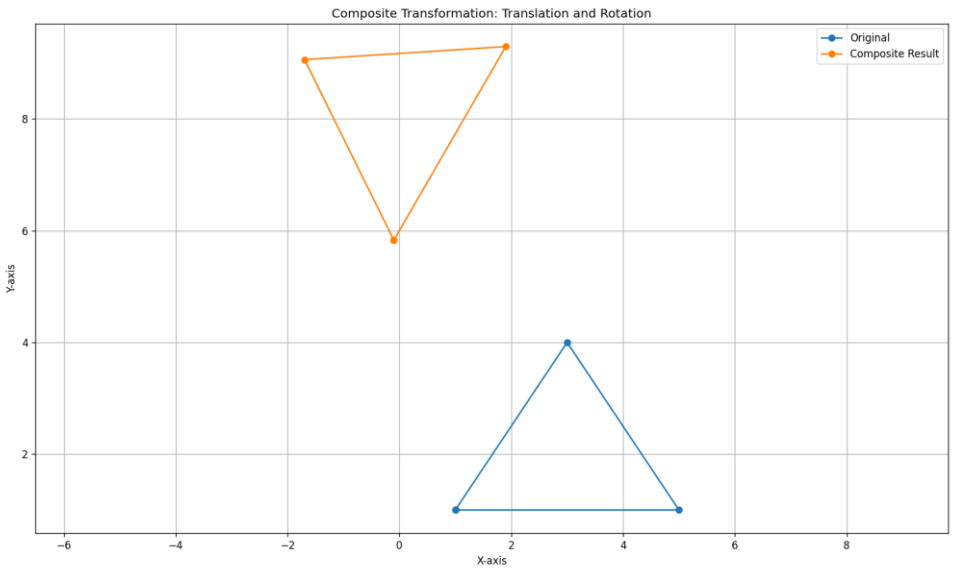

Figure 9: Output of Composite (Translation, Rotation) Transformation

Experiment No. 10

Name of the Problem: Cohen-Sutherland Line Clipping Algorithm

Source Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

INSIDE = 0
LEFT = 1
RIGHT = 2
BOTTOM = 4
TOP = 8

x_min, y_min = 2, 2
x_max, y_max = 12, 8
x1, y1 = 0, 4
x2, y2 = 15, 10
original = (x1, y1, x2, y2)

def compute_code(x, y):
    code = INSIDE
    if x < x_min:
        code |= LEFT
    elif x > x_max:
        code |= RIGHT
    if y < y_min:
        code |= BOTTOM
    elif y > y_max:
        code |= TOP
    return code

code1 = compute_code(x1, y1)
code2 = compute_code(x2, y2)
accept = False

while True:
    if code1 == 0 and code2 == 0:
        accept = True
        break
    elif code1 & code2:
        break
    else:
        code_out = code1 if code1 != 0 else code2
        if code_out & TOP:
            x = x1 + (x2 - x1) * (y_max - y1) / (y2 - y1)
            y = y_max
        elif code_out & BOTTOM:
            x = x1 + (x2 - x1) * (y_min - y1) / (y2 - y1)
            y = y_min
        elif code_out & RIGHT:
            y = y1 + (y2 - y1) * (x_max - x1) / (x2 - x1)
            x = x_max
        else:
            y = y1 + (y2 - y1) * (x_min - x1) / (x2 - x1)
            x = x_min

        if code_out == code1:
            x1, y1 = x, y
            code1 = compute_code(x1, y1)
        else:
            x2, y2 = x, y
            code2 = compute_code(x2, y2)

plt.figure(figsize=(16, 9))
plt.gca().add_patch(Rectangle((x_min, y_min), x_max - x_min, y_max - y_min, fill=False, linewidth=3))
plt.plot([original[0], original[2]], [original[1], original[3]], linestyle="--", marker="o", label="Original Line")
if accept:
    plt.plot([x1, x2], [y1, y2], marker="o", linewidth=4, label="Clipped Line")
plt.title("Cohen-Sutherland Line Clipping Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.savefig("10_cohen_sutherland_output.png", dpi=120)
plt.show()
```

Output Screenshot:

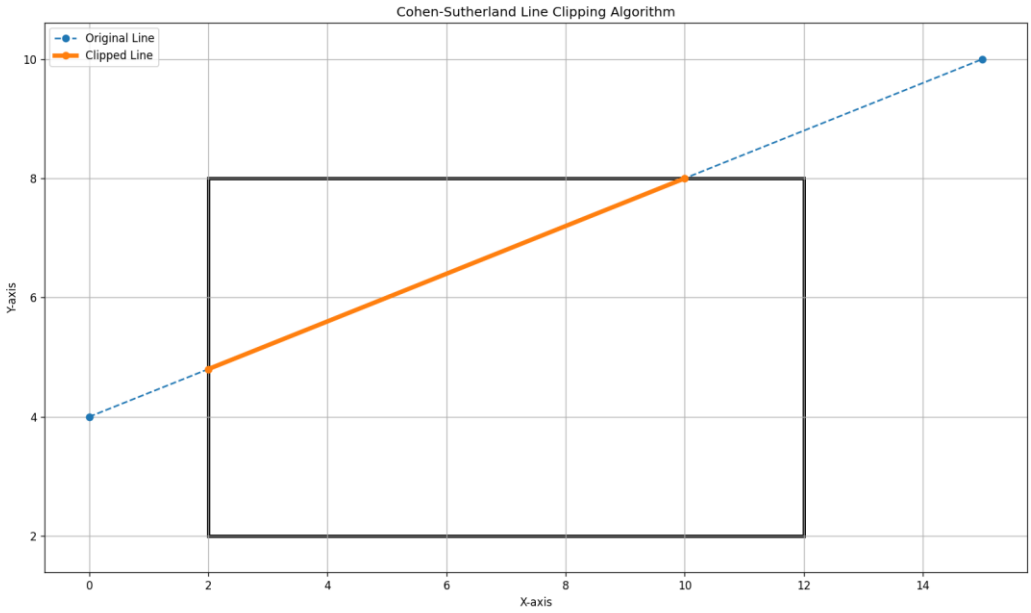

Figure 10: Output of Cohen-Sutherland Line Clipping Algorithm

Experiment No. 11

Name of the Problem: Sutherland-Hodgman Polygon Clipping Algorithm

Source Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

polygon = [(1, 1), (12, 3), (10, 10), (4, 8)]
x_min, y_min = 3, 3
x_max, y_max = 9, 7

def inside(p, edge):
    x, y = p
    if edge == "left":
        return x >= x_min
    if edge == "right":
        return x <= x_max
    if edge == "bottom":
        return y >= y_min
    return y <= y_max

def intersection(s, e, edge):
    x1, y1 = s
    x2, y2 = e
    if edge == "left":
        x = x_min
        y = y1 + (y2 - y1) * (x_min - x1) / (x2 - x1)
    elif edge == "right":
        x = x_max
        y = y1 + (y2 - y1) * (x_max - x1) / (x2 - x1)
    elif edge == "bottom":
        y = y_min
        x = x1 + (x2 - x1) * (y_min - y1) / (y2 - y1)
    else:
        y = y_max
        x = x1 + (x2 - x1) * (y_max - y1) / (y2 - y1)
    return (x, y)

def clip(poly, edge):
    output = []
    s = poly[-1]
    for e in poly:
        if inside(e, edge):
            if inside(s, edge):
                output.append(e)
            else:
                output.append(intersection(s, e, edge))
                output.append(e)
        elif inside(s, edge):
            output.append(intersection(s, e, edge))
        s = e
    return output

clipped = polygon
for edge in ["left", "right", "bottom", "top"]:
    clipped = clip(clipped, edge)

def draw(poly, label):
    closed = poly + [poly[0]]
    xs = [p[0] for p in closed]
    ys = [p[1] for p in closed]
    plt.plot(xs, ys, marker="o", label=label)

plt.figure(figsize=(16, 9))
plt.gca().add_patch(Rectangle((x_min, y_min), x_max - x_min, y_max - y_min, fill=False, linewidth=3))
draw(polygon, "Original Polygon")
draw(clipped, "Clipped Polygon")
plt.title("Sutherland-Hodgman Polygon Clipping Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.savefig("11_sutherland_hodgman_output.png", dpi=120)
plt.show()
```

Output Screenshot:

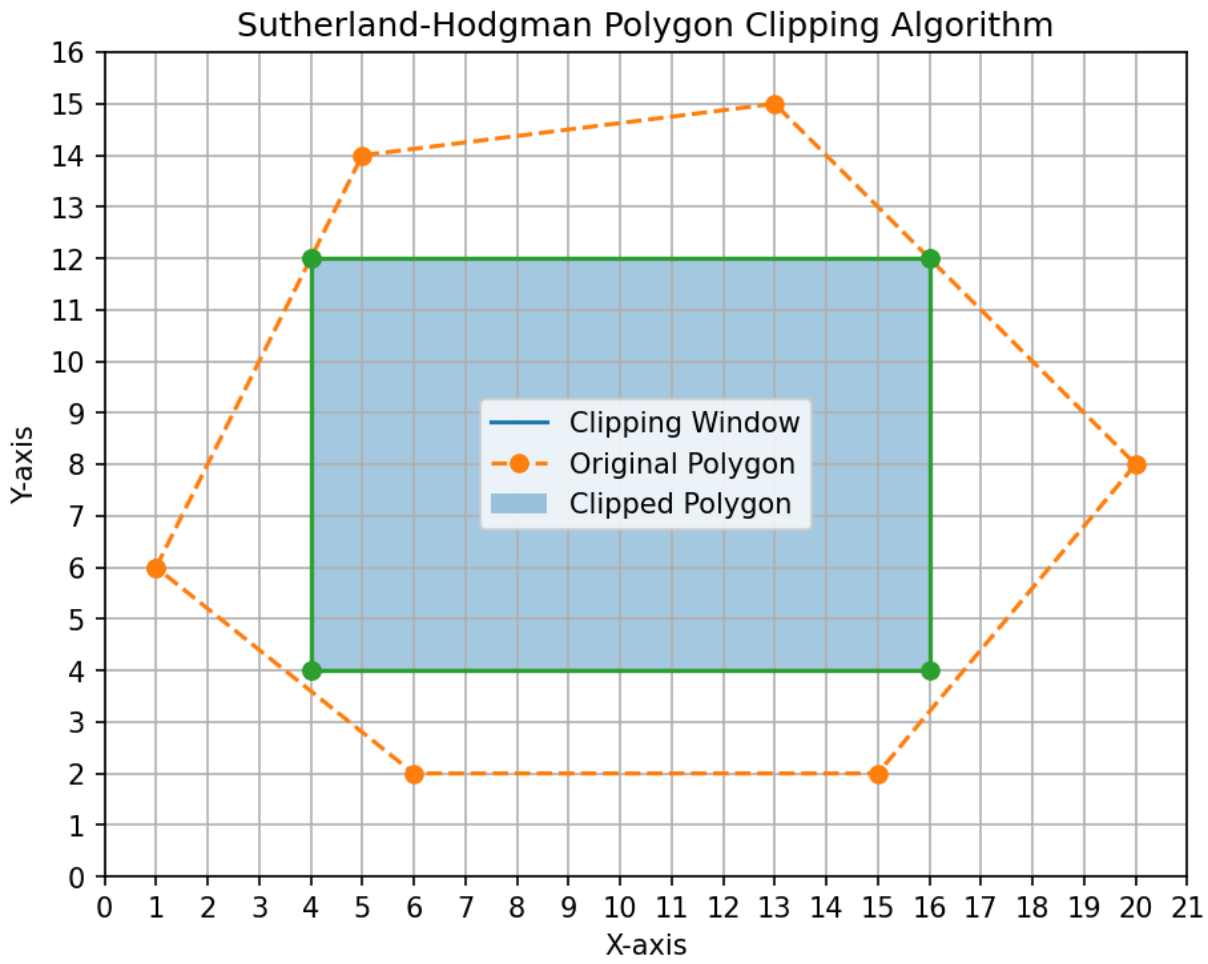

Figure 11: Output of Sutherland-Hodgman Polygon Clipping Algorithm

Experiment No. 12

Name of the Problem: General Point-to-Point Rotation of a Triangle

Source Code:

```
import math
import matplotlib.pyplot as plt

triangle = [(2, 1), (7, 1), (4, 5), (2, 1)]
px, py = 4, 2
angle = 60
rad = math.radians(angle)

rotated = []
for x, y in triangle:
    x_shift = x - px
    y_shift = y - py
    xr = x_shift * math.cos(rad) - y_shift * math.sin(rad) + px
    yr = x_shift * math.sin(rad) + y_shift * math.cos(rad) + py
    rotated.append((xr, yr))

def draw(poly, label):
    xs = [p[0] for p in poly]
    ys = [p[1] for p in poly]
    plt.plot(xs, ys, marker="o", label=label)

plt.figure(figsize=(16, 9))
draw(triangle, "Original Triangle")
draw(rotated, "Rotated Triangle")
plt.scatter([px], [py], s=160, label="Rotation Point")
plt.title("General Point to Point Rotation of a Triangle")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.legend()
plt.savefig("12_point_rotation_triangle_output.png", dpi=120)
plt.show()
```

Output Screenshot:

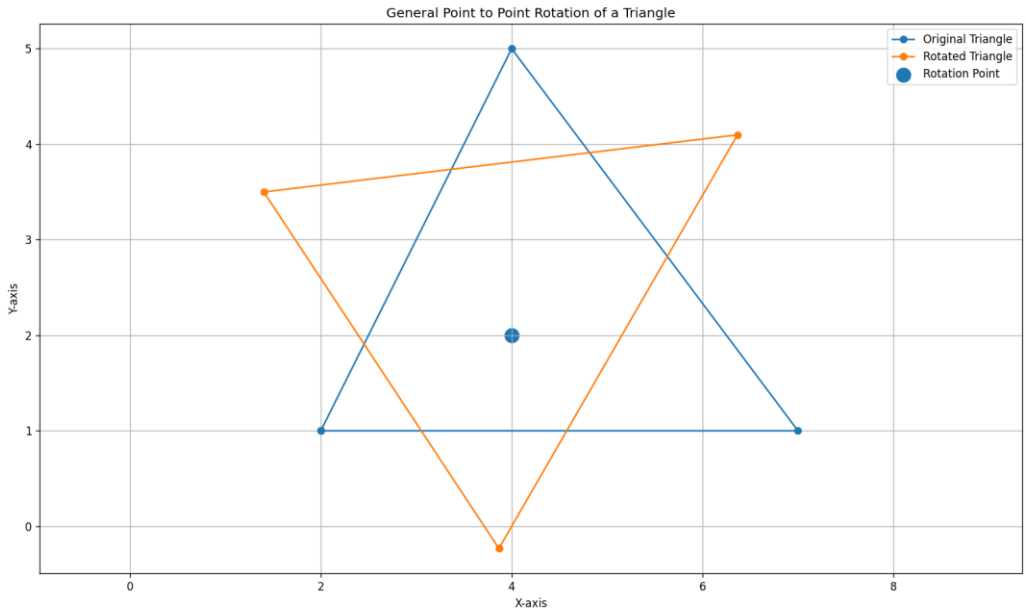

Figure 12: Output of General Point-to-Point Rotation of a Triangle